

Contents

1	Math	1
1.1	Fpow	1
1.2	Combination	1
1.3	Extgcd	1
2	Shortest Path	1
2.1	Dijkstra	1
2.2	Bellman-Ford	1
2.3	Floyd-Warshall	1
3	DP	1
3.1	0-1 Knapsack	1
3.2	Unbounded Knapsack	2
3.3	SOS DP	2
3.4	O(lgn) LIS	2
4	Graph	2
4.1	Euler Path	2
5	Data Structure	2
5.1	Segment Tree	2
5.2	Binary Indexed Tree	2
6	Others	3
6.1	Discretization	3

1 Math

1.1 Fpow

```
int fpow(int x, int n, int mod) {
    int res = 1;
    while (n) {
        if (n & 1) res = res * x % mod;
        x = x * x % mod;
        n >>= 1;
    }
    return res;
}
```

1.2 Combination

```
#define MXN 1'000'005
#define N 1'000'000
#define ll long long

ll fac[MXN], inv[MXN];
fac[0] = 1; // 0! = 1
for(int i = 1; i <= N; i++) fac[i] = fac[i-1] * i % mod;
inv[N] = fpow(fac[N], mod-2); // 快速幂
for(int i = N-1; i >= 0; i--) inv[i] = inv[i+1] * (i+1) % mod;

ll C(int n, int k) {
    return fac[n] * inv[k] % mod * inv[n-k] % mod;
}
```

1.3 Extgcd

```
#define ll long long
int extgcd(int a, int b, int &x, int &y) {
    if(b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    int GCD = extgcd(b, a % b, y, x);
    y -= a / b * x;
    return GCD;
}

int main() {
    int a, b, x, y;
    cin >> a >> b;
    int GCD = extgcd(a, b, x, y);
}
```

2 Shortest Path

2.1 Dijkstra

```
#define pii pair<int,int>
#define INF 1e18
vector<pair<int,int>>vec[N];
void dijkstra(int s, int t) { // 起點, 終點
    int dis[N];
    for(int i=0; i<N; i++) {
        dis[i] = INF; // 值要設為比可能的最短路徑權重還要大的值
    }
    dis[s] = 0;
    priority_queue<pii, vector<pii>, greater<pii>> pq;
    pq.push({dis[s], s});
    while(pq.empty() == 0) {
        int u = pq.top().second;
        pq.pop();
        if(vis[u]) continue;
        vis[u] = 1;
        for(auto [v, w] : vec[u]) {
            if(dis[u] + w < dis[v]) {
                dis[v] = dis[u] + w;
                pq.push({dis[v], v});
            }
        }
    }
}
```

2.2 Bellman-Ford

```
struct edge {
    int v, w;
};
int s, n; // 起點 點數量
vector<edge> E[MXN];
vector<int> dis(MXN, INF), cnt(MXN), inq(MXN);
// cnt記錄最短路經過幾條邊
// inq記錄節點是否在queue裡面
queue<int> Q;
dis[s] = 0; Q.push(s); inq[s] = 1;
while(Q.size()) {
    int u = Q.front(); Q.pop();
    inq[u] = 0;
    for(auto [v, w] : E[u]) {
        if(dis[v] > dis[u] + w) {
            dis[v] = dis[u] + w;
            cnt[v] = cnt[u] + 1;
            if(cnt[v] >= n) {有負環}
            if(!inq[v]) Q.push(v), inq[v] = 1;
        }
    }
}
```

2.3 Floyd-Warshall

```
for(int k=1; k<=N; k++) { // 窮舉中繼點k
    for(int i=1; i<=N; i++) {
        for(int j=1; j<=N; j++) { // 窮舉點對(i,j)
            dis[i][j] = min(dis[i][j], dis[i][k] + dis[k][j]);
        }
    }
}
```

3 DP

3.1 0-1 Knapsack

```
for(int i = 1; i <= n; ++i) {
    for(int w = W; w >= 0; --w)
        dp[w] = max(w - c[i] >= 0 ? dp[w - c[i]] + v[i] : 0, dp[w]);
}

// Backtracing
for(int i = 1; i <= n; ++i) {
    for(int w = 0; w <= W; ++w) {
        // dp[i][w] 考慮前 i 個物品且負重上限為 w 的答案。
    }
}
```

```

    int a = w - c[i] >= 0 ? dp[i - 1][w - c[i]] + v
    [i] : 0;
    int b = dp[i-1][w];
    if(a > b){
        dp[i][w] = a;
        R[i][w] = w-c[i];
    }else{
        dp[i][w] = b;
        R[i][w] = w;
    }
}
}

int i = n, w = W;
vector<int> ans;
while(i){
    if(w != R[i][w]) ans.push_back(i); //不一樣代表有取
    w = R[i][w];
    --i;
}
}

```

3.2 Unbounded Knapsack

```

for(int w = 0; w <= W; ++w){
    for(int i = 1; i <= n; ++i){
        dp[w] = max(dp[w], w-c[i] >= 0 ? dp[w-c[i]] + v
        [i] : 0);
    }
}

```

3.3 SOS DP

```

for(int i = 0; i < (1<<N); ++i)
    F[i] = A[i];
for(int i = 0; i < N; ++i)
    for(int mask = 0; mask < (1<<N); ++mask)
        if(mask & (1<<i))
            F[mask] += F[mask^(1<<i)];
//窮舉子集合
for(int s = mask; s ; s = (s-1)&mask;)

```

3.4 O(lgn) LIS

```

#define all(v) (v).begin(), (v).end()

vector<int> dp;
for(int i = 0; i < n; ++i){
    auto it = lower_bound(all(dp), A[i]);
    if(it == dp.end()){
        dp.push_back(A[i]);
    }else{
        *it = A[i];
    }
}
cout << dp.size() << '\n';

```

4 Graph

4.1 Euler Path

```

vector<int> path;
void dfs(int x){
    while(!edge[x].empty()){
        int u = edge[x].back();
        edge[x].pop_back();
        dfs(u);
    }
    path.push_back(x);
}

int main(){
    dfs(st);
    reverse(path.begin(), path.end());
}

```

```

for(int i:path)    cout<<i<<' ';
}

```

5 Data Structure

5.1 Segment Tree

```

#define cl(x) (x<<1)
#define cr(x) (x<<1)+1
const int N;
int seg[4*N];
int arr[N];
void pull(int id){
    seg[id]=max(seg[cl(id)], seg[cr(id)]);
}
void build(int id, int l, int r){
    if(l==r){
        seg[id]=arr[l];
        return ;
    }
    int mid=(l+r)>>1;
    build(cl(id), l, mid);
    build(cr(id), mid+1, r);
    pull(id);
}
void update(int id, int l, int r, int x, int v){
    if(l==r){
        seg[id]=v;
        return ;
    }
    int mid=(l+r)>>1;
    if(x<=mid){
        update(cl(id), l, mid, x, v);
    }
    if(mid<x){
        update(cr(id), mid+1, r, x, v);
    }
    pull(id);
}
int query(int id, int l, int r, int sl, int sr){
    if(sl<=l&&r<=sr){//目前這個區間在查詢區間內
        return seg[id];
    }
    int mid=(l+r)>>1, res=0;
    if(sl<=mid){//左區間跟查詢區間有交集
        res=max(res, query(cl(id), l, mid, sl, sr));
    }
    if(mid<sr){//右區間跟查詢區間有交集
        res=max(res, query(cr(id), mid+1, r, sl, sr));
    }
    return res;
}

```

5.2 Binary Indexed Tree

```

struct Binary_Indexed_Tree{
    int n;
    vector<long long> bit;
    int lowbit(int x){
        return x&-x;
    }
    void init(int _n){
        n = _n+1;
        bit = vector<long long>(n, 0);
    }
    void update(int x, int v){
        for(; x<n; x+=lowbit(x)){
            bit[x] += v;
        }
    }
    long long query(int x){
        long long ret = 0;
        for(; x>0; x-=lowbit(x)){
            ret += bit[x];
        }
        return ret;
    }
}

```

```
    }  
}BIT;
```

6 Others

6.1 Discretization

```
//vector<int> arr;  
vector<int> tmp(arr); //將arr複製到tmp  
sort(tmp.begin(), tmp.end());  
tmp.erase(unique(tmp.begin(), tmp.end()), tmp.end());  
for (int i = 0; i < n; i++)  
    arr[i] = lower_bound(tmp.begin(), tmp.end(), arr[i]  
        ) - tmp.begin();
```